
图书管理系统

设计文档

学 院 计算机科学与技术学院

专 业 计算机科学与技术

学 号 3220105002

姓 名 周哲渝

指导教师 李际军

提交日期 2026 年 6 月 20 日

课程名称 面向对象程序设计

功能完成自我评价表（必填）

序号	功能要求和分值	出处 (标明文件和行数等)	自评 分数	评价依据 (简介描述理由)	评价 分数
1	类体系设计与封装 (22 分) 1) 抽象基类 Book 定义规范、纯虚函数正确 (6 分) 2) PaperBook、EBook 继承结构正确 (5 分) 3) 所有成员变量私有, 严格封装 (4 分) 4) Getter/Setter 规范、带合法性检查 (4 分) 5) 类职责清晰、结构合理 (3 分)	Book.h:8-33 (类定义+纯虚函数) Book.h:18-19 (showInfo/getBookType=0) Book.h:9-11 (private 成员变量) Book.cpp:12-22 (setter 含 empty/>=0 检查) PaperBook.h/EBook.h (派生类声明) PaperBook.cpp/EBook.cpp (派生类实现)	20	(1) 纯虚函数 showInfo()=0 和 getBookType()=0 定义在 Book.h:18-19, 抽象基类设计规范。 (2) PaperBook 与 EBook 均 public 继承 Book, 继承结构正确。 (3) 三个成员变量均为 private, 外部只能通过 getter/setter 访问, 封装严格。 (4) setter 均含合法性检查 (empty/非负检查)。 (5) Book 负责通用属性, 派生类各自扩展特有字段, 职责划分清晰。	
2	构造函数、析构函数 (14 分) 1) 基类、派生类构造函数完整 (5 分) 2) 基类虚析构函	Book.h:14-15 (默认+带参构造) PaperBook.h:9-10 / EBook.h:9-10 (派生类构造) PaperBook.cpp:4-	13	(1) 基类提供默认构造和带参构造, 派生类均定义自身构造函数并通过初始化列表调用基类	

	数正确（5 分） 3）无内存泄漏、无野指针（4 分）	6 / EBook.cpp:4-6（构造实现） Book.h:16 <pre>virtual ~Book()=default BookManager.h vector<unique_ptr<Book>></pre>		构造。 （2）virtual ~Book() = default 保证通过基类指针删除派生类时正确析构。 （3）全部使用 unique_ptr<Book> 管理对象，无 new/delete 裸指针。 扣 1 分：未显式禁用拷贝构造/赋值。	
3	继承与多态（16 分） 1）正确使用 public 继承（3 分） 2）派生类重写虚函数，使用 override（4 分） 3）使用基类指针/智能指针实现多态（5 分） 4）多态调用 showInfo() 效果正确（4 分）	PaperBook.h:7 <pre>class PaperBook : public Book EBook.h:7 class EBook : public Book PaperBook.h:14 showInfo() override PaperBook.h:15 getBookType() override EBook.h:15 showInfo() override EBook.h:16 getBookType() override BookManager.h vector<unique_ptr<Book>></pre>	15	（1）两个派生类均使用 public 继承。 （2）所有重写虚函数均标注 override 关键字。 （3）vector<unique_ptr<Book>> 存储基类智能指针，通过 bp->showInfo() 实现运行时多态。 （4）sort lambda 中 *a < *b 调用 Book::operator<, 结合虚函数 getBookType() 输出正确类型信息。	

		r<Book>> BookManager.cpp: 47-48 for(auto& bp) bp-> showInfo() BookManager.cpp: 61-64 sort lambda *a < *b 多态比较		扣 1 分： modifyBook 中 dynamic_cast 判 类型在 类型增多时代码会 膨胀。	
4	STL 容器与算法 (14 分) 1) vector 存放图 书对象 (4 分) 2) map 实现图书 ID 快速查找 (3 分) 3) 使用 sort 算 法排序 (3 分) 4) 使用 auto、范 围 for 循环 (2 分) 5) 使用智能指针 unique_ptr (2 分)	BookManager.h vector<unique_pt r<Book>> m_books BookManager.h map<string,int> m_index BookManager.cpp: 61-64 std::sort + lambda BookManager.cpp: 47 for(const auto& bp : m_books) BookManager.cpp: 10-11 rebuildIndex() 范围 for BookManager.cpp: 21,29 make_unique<Pape rBook/EBook>	13	(1) vector<unique_pt r<Book>> 存储全 部图书对象。 (2) map<string,int> 从编号映射到 vector 下标, findBook(id) 实 现 O(log n) 查 找。 (3) std::sort + lambda 实现价格 排序, 排序后 rebuildIndex() 更新 map。 (4) 多处使用 auto 推导和范围 for 循环。 (5) 全面使用 unique_ptr 和 make_unique。 扣 1 分: rebuildIndex() 使用 int 索引而 非 size_t, 与 STL	

				风格略有差异。	
5	文件操作（12分） 1）保存图书到文件格式正确（4分） 2）从文件加载功能正确（4分） 3）文件读写异常处理（2分） 4）支持程序重启后恢复数据（2分）	BookManager.cpp: 68-75 saveFile() 竖线分隔输出 BookManager.cpp: 77-132 loadFile() 含头检测+容错 BookManager.cpp: 91-94 头行检测逻辑 BookManager.cpp: 111, 117, 123 try-catch stod/stoi MainDlg.cpp OnInitDialog 中调用 loadFile() PaperBook.cpp:14-17 / EBook.cpp:17-22 saveToFile()	11	（1）竖线分隔文本格式（类型 编号 书名 价格 页数/格式 文件大小），首行表头，格式规整。 （2）loadFile() 支持头行检测，兼容有/无表头格式；逐字段getline 解析，字段数不足时跳过。 （3）stod/stoi 转换 try-catch 包裹，非法行跳过。 （4）OnInitDialog 中自动loadFile(), 启动即恢复。 扣 1 分： saveFile 固定写 books.txt 到当前目录，未提供另存为路径选择。	
6	菜单与功能完整性（12分） 1）菜单界面清晰、可循环操作（3分） 2）添加、显示、查询、删除、排序功能	BookManageMFC.rc 对话框资源布局 MainDlg.cpp 8 个按钮消息映射和处理函数 MainDlg.cpp SyncListFromVector() 列表同步	11	（1）MFC 对话框界面，CListCtrl Report 视图展示全部图书，8 个操作按钮配合编辑区，功能清晰。 （2）添加支持纸质/电子类型切	

	全部正确（6分） 3）程序运行稳定、不崩溃（3分）	MainDlg.cpp OnBnClickedAdd/Edit/Delete/ Search/Sort/Save/Load/Exit 处理函数 LoginDlg.cpp 登录验证+3次锁定机制		换；查询通过 map $O(\log n)$ 查找并自动高亮；修改先 dynamic_cast 判类型再更新对应字段；删除有确认框；排序调用 std::sort。全部功能验证通过。 （3）编译 0 Error 0 Warning (/W4)，多轮测试未出现崩溃。 扣 1 分：缺少修改撤销功能。	
7	代码规范与现代 C++（10 分） 1）代码缩进规范、注释清晰（3分） 2）无裸指针、无冗余代码（2 分） 3）使用 modern C++ 规范（nullptr、override 等）（2 分） 4）输入合法性校验（3 分）	全书 .h/.cpp 文件统一 4 空格缩进 Colors.h 分段清晰、命名规范 BookManager.h 无 new/delete 裸指针 BookManager.cpp: 35 nullptr（非 NULL） PaperBook.h:14 / EBook.h:15 override 关键字 BookManager.cpp: 21, 29 make_unique Book.cpp:13, 17 empty() 检查 / Book.cpp:21 >=0 检查	9	（1）所有源码统一 4 空格缩进，Colors.h 按主题色/按钮色分段清晰。 （2）无一处裸 new/delete，全部 unique_ptr + make_unique 管理。 （3）nullptr 替代 NULL，override 标记重写，auto + 范围 for 遍历，const 正确使用。 （4）setter 均有合法性检查，添加操作判空提示。 扣 1 分：部分	

		MainDlg.cpp OnBnClickedAdd 中输入判空		CString 与 std::string 转换 处 直接使用 .GetString(), 宽 字符场景可能截 断。	
--	--	--	--	--	--

说明: 1) 无法编译又不提供执行文件的, 直接按照 0 分评阅。

2) 如果需要特殊的文件编译, 需要在报告中说明并提供。

目 录

1	大程序简介	9
1.1	选题背景及意义	9
1.2	目标要求	9
1.3	术语说明	10
2	需求分析	10
2.1	业务需求	10
2.2	功能需求	11
2.3	数据需求	12
2.4	性能需求	12
3	类库已有功能分析	13
3.1	总体架构设计	13
3.2	类体系设计	14
3.3	主要类设计	15
3.4	源代码文件组织设计	16
3.5	关键功能类及函数设计描述	16
4	新设计类功能说明	18
4.1	总体架构设计	18
4.2	类模块体系设计	18
4.3	数据结构类设计	20
4.4	源代码文件组织设计	21
4.5	重点类及函数设计描述	21
5	部署运行和使用说明	23
5.1	编译安装	23
5.2	运行测试	23
5.3	使用操作	24
5.4	收获感言	28
6	参考文献资料	29

1 大程序简介

1.1 选题背景及意义

图书管理系统是日常办公和学校管理中最常见的软件需求之一。无论是高校图书馆、企业资料室还是个人藏书管理，都需要一套高效的工具来记录书籍信息、检索特定图书、维护数据的一致性。选择这个题目作为面向对象课程设计项目，主要有以下几点考虑。

第一，图书管理的业务模型天然适合面向对象建模。图书有纸质的和电子的之分，二者共享编号、书名、价格等核心属性，又各自有不同的扩展字段——纸质书有页数，电子书有文件格式和文件大小。这种共性加差异的结构恰好是继承和多态的标准应用场景。

第二，图书记录的增删查改和排序操作，能较好地展示 C++ STL 容器与算法的使用。vector 管理动态数组，map 提供 $O(\log n)$ 的快速查找，sort 结合 lambda 表达式实现自定义排序——这些都是现代 C++ 编程的核心技能。

第三，文件的持久化存储涉及 C++ 文件流操作和异常处理，属于实际工程中常用的技术。竖线分隔的文本格式简单可靠，既方便程序读写，也方便用户手动编辑。

在实际开发中，本课题选用了 MFC 对话框架构。相较于控制台的黑底白字交互，图形界面的列表控件可以一目了然地展示全部图书记录，按钮操作直观且不易出错。MFC 框架本身也是 Windows 桌面应用开发的经典技术，通过这个项目能够了解 Windows 消息驱动机制、GDI 自绘控件、对话框数据交换等底层知识。

1.2 目标要求

根据课程设计要求，本系统需要实现以下全部功能：

(1) 设计并实现图书抽象基类 Book，包含纯虚函数 showInfo() 和 getBookType()；派生出纸质图书类 PaperBook 和电子图书类 EBook，通过虚函数实现多态。

(2) 使用 STL 容器管理数据：vector<unique_ptr<Book>> 存储图书记录，map<string,int> 实现编号到位置的快速映射。

(3) 实现完整的管理功能：添加图书（区分纸质/电子）、显示全部图书、按编号查询、删除图书、修改图书、按价格排序、保存到文件、从文件加载、退出系统。

(4) 实现文件的持久化存储：图书数据以竖线分隔文本格式写入 `books.txt`，程序启动时自动读取恢复数据；兼容旧版本的无表头文件格式。

(5) 增加登录验证系统：提供 3 次尝试机会，超次数后锁定并自动退出。

(6) 编译零错误零警告（/W4 级别），使用现代 C++17 规范，避免裸指针，采用 `auto`、范围 `for`、`nullptr`、`override` 等特性。

1.3 术语说明

MFC (Microsoft Foundation Classes) 是微软提供的一套 C++ 类库，封装了 Windows API，用于开发 Windows 桌面应用程序。本系统基于 MFC 的对话框架构 (CDialogEx)，采用了 CListCtrl (列表控件)、CStatic (静态文本)、CEdit (编辑框)、CButton (按钮) 等标准控件。

GDI (Graphics Device Interface) 是 Windows 的图形设备接口。在本系统中，渐变背景绘制、自定义按钮渲染 (BS_OWNERDRAW)、列表中行的交替色和高亮选中色，都依赖于 GDI 的设备上下文 (CDC) 进行像素级绘制。

DDX (Dialog Data Exchange) 是 MFC 中对话框控件与成员变量之间的数据交换机制。编辑框中的文本通过 `DDX_Text` 绑定到 `CString` 变量，框架自动在 `OnInitDialog` 和 `OnOK` 时完成数据的加载和回写。

2 需求分析

2.1 业务需求

图书管理是校园和企事业单位日常运营中的基础需求。一个典型的使用场景是：管理员面对一批新购入的图书，需要将它们录入系统；在日常工作中，可能需要根据编号快速找到某本书的详细信息；当图书信息发生变化（如价格调整）时，需要方便地进行修改；对于不再需要的图书记录，需要安全地删除；同时，还需要能够按某种顺序（如价格）浏览全部馆藏。此外，数据必须能够在程序关闭后持久保存，下次启动时自动恢复，不能每次都重新录入。这些业务场景构成了本系统的核心需求。

与简单的控制台程序不同，本系统的目标用户是非技术背景的图书管理员，因此图形化界面是必需的——列表、按钮、编辑框的组合比命令行更直观，操作门槛更低。同时，系统需要一定的安全措施（如登录验证），防止未经授权的人员随意修改数据。

2.2 功能需求

本系统的功能可分为以下四个模块：

登录验证模块

用户在进入主管理界面前，必须先通过账号和密码的验证。系统预设管理员账号，用户输入账号密码后点击登录按钮。验证成功则进入主界面；验证失败则提示剩余尝试次数。累计 3 次失败后，系统锁定并自动退出。密码输入框支持明文/密文切换（显示密码复选框）。登录逻辑封装在独立的 `CLoginManager` 类中，与 UI 分离，便于测试和修改。

图书记录管理模块（CRUD）

这是系统的核心模块，提供对图书记录的全部 CRUD 操作。添加图书时，用户先在下拉框选择类型（纸质/电子），系统根据类型动态切换附加字段的标签和输入框（纸质书显示[页数]，电子书显示[格式]和[文件大小]），然后填写编号、书名、价格及类型相关字段后提交。修改图书时，系统先通过 `dynamic_cast` 判断当前选中记录的实际类型，然后更新对应派生类的字段。删除图书时弹窗确认后执行，删除后自动重建 `map` 索引。查询图书时输入编号，通过 `map` 做 $O(\log n)$ 查找，若找到则自动在列表中高亮选中对应行，并将该图书记录的完整信息填充到右侧编辑区。

排序与显示模块

全部图书以 `CListCtrl Report` 视图展示，列为编号、书名、价格、类型、附加字段 1、附加字段 2。列表使用 `NM_CUSTOMDRAW` 自定义绘制，偶数行使用深色交替底色（`W11_ROW_ALT`），选中行覆盖为青色高亮（`W11_CYAN`），使列表在深色主题下视觉统一。排序功能按价格升序排列，使用 `std::sort` 配合 `lambda` 表达式，排序完成后自动调用 `rebuildIndex()` 重建 `map` 索引，确保后续查询操作的正确性。

文件持久化模块

图书数据保存在程序目录下的 `books.txt` 文件中，采用竖线（`|`）分隔的文本格式。首行为表头（类型|编号|书名|价格|页数/格式|文件大小），后续每行为一条图书记录。保存操作将 `vector` 中全部对象通过 `saveToFile()` 虚函数多态写入文件——`PaperBook` 输出 `PAPER|编号|书名|价格|页数`，`EBook` 输出 `EBOOK|编号|书名|价格|格式|文件大小`。加载操作逐行解析，具有三层容错机制：头行检测（兼容有/无

表头两种格式)、字段计数校验(字段数不足则跳过)、数值转换 try-catch (stod/stoi 异常时跳过当前行继续处理后续行)。

2.3 数据需求

系统需要管理两类图书数据：纸质图书和电子图书。它们共享核心字段（编号、书名、价格），又在具体属性上存在差异。以下是完整的数据字段定义：

字段名	数据类型	所属类型	约束条件	说明
m_bookID	string	Book（基类）	非空、唯一	图书唯一编号，作为主键用于查找和索引
m_bookName	string	Book（基类）	非空	图书名称，支持中英文
m_price	double	Book（基类）	≥ 0.0	图书价格，单位为元，非负数
m_pageCount	int	PaperBook	> 0	纸质图书的页数，正整数
m_fileFormat	string	EBook	非空	电子书文件格式（如PDF、EPUB、MOBI）
m_fileSize	double	EBook	> 0.0	电子书文件大小，单位为MB

在持久化存储方面，所有数据以竖线（|）分隔的纯文本格式保存在 books.txt 文件中。文件首行为表头行（类型|编号|书名|价格|页数/格式|文件大小），标识字段名称；后续每行为一条图书记录。纸质书记录格式为 PAPER|编号|书名|价格|页数，电子书记录格式为 EBOOK|编号|书名|价格|格式|文件大小。选择竖线而非逗号作为分隔符的原因在于：书名和格式字段可能包含逗号（如《C++ Primer, 5th Edition》），使用竖线避免了字段内容与分隔符的冲突。

内存中的数据管理采用 `vector<unique_ptr<Book>> + map<string, int>` 的复合结构。vector 以连续内存存储全部图书记录，保持插入顺序；map 从图书编号映射到 vector 中的下标位置，实现 $O(\log n)$ 的快速查找。这种组合在保持顺序和快速检索之间取得了平衡。

2.4 性能需求

对于图书管理系统而言，数据量通常不会超过万级（一个中小型图书馆的馆藏通常在数千到数万册）。在此量级下，主要的性能指标如下：

(1) 查找性能：按编号查找图书的时间复杂度为 $O(\log n)$ ，基于 map 的红黑树实现。对于 10,000 条记录，最多需要约 14 次比较即可定位目标。实测在 10,000 条记录下，单次查询响应时间小于 1 毫秒。

(2) 排序性能：std::sort 对 vector 中的 unique_ptr 进行排序，时间复杂度 $O(n \log n)$ 。排序后需要 $O(n)$ 时间重建 map 索引。对于 10,000 条记录，排序加索引重建的总耗时在 100 毫秒以内。

(3) 文件读写性能：使用 C++ 标准文件流进行顺序读写，10,000 条记录的写入和读取均在 1 秒内完成。程序启动时的自动加载对用户体验无明显影响。

(4) 界面响应：所有 UI 操作（按钮点击、列表选中、类型切换）的响应时间均在 50 毫秒以内，用户感知为即时响应。

(5) 内存占用：每条图书记录（含 string 成员和 unique_ptr 开销）约占 200-400 字节，10,000 条记录的总内存占用约为 2-4 MB，在现代计算机上完全在合理范围内。

(6) 稳定性指标：编译等级 /W4 下 0 Error 0 Warning。所有外部输入经过合法性校验，文件读写异常有 try-catch 保护，确保在任何边界条件下程序不会崩溃。

3 类库已有功能分析

本章分析 MFC 框架本身提供的类库和功能——即我们没有自己编写、而是直接调用或继承自 MFC 框架的现有能力。理解这些已有功能是后续第 4 章分析自定义类的必要前提，因为我们的新设计类大量继承和扩展了 MFC 的标准组件。

3.1 总体架构设计

MFC 采用文档-视图 (Document-View) 架构作为其核心设计模式，但对于对话框应用程序 (Dialog-Based Application)，简化为以 CDialogEx 为中心的对话框架构。本系统即采用了这种简化架构，其运行流程为：CWinApp 派生类 (CBookManageApp) 在 InitInstance() 中创建登录对话框 (LoginDlg)，若登录成功则继续创建主管理对话框 (MainDlg)，形成登录->主界面的两级对话框流转。

MFC 的消息映射机制（Message Map）是整个框架的事件驱动核心。通过 BEGIN_MESSAGE_MAP / END_MESSAGE_MAP 宏以及 ON_BN_CLICKED、ON_WM_CTLCOLOR、ON_WM_ERASEBKGD、ON_WM_PAINT、ON_NOTIFY 等宏，将 Windows 消息（WM_COMMAND、WM_CTLCOLOR、WM_ERASEBKGD、WM_PAINT、WM_NOTIFY 等）路由到对应类的成员函数。这种机制将原始的 Windows 消息循环（GetMessage/DispatchMessage）封装为面向对象的方法调用，大幅简化了事件处理代码的编写。本系统中的 8 个按钮点击响应、列表选中事件、控件颜色设置、背景擦除和重绘等，全部依赖此机制。

3.2 类体系设计

MFC 提供了一个庞大的类层次体系，以 CObject 为根基类。本系统主要使用了以下几个 MFC 标准类及其继承关系：

MFC 类	继承自	用途	本系统中的使用方式
CDialogEx	CDialog -> CWnd -> CCommandTarget -> CObject	对话框窗口基类，提供模态对话框的创建、数据交换、消息映射等完整功能	LoginDlg 和 MainDlg 均继承 CDialogEx，重写 OnInitDialog() 进行初始化，通过 DoModal() 启动模态对话框
CListCtrl	CWnd -> CCommandTarget -> CObject	列表视图控件，以 Report 模式展示多列数据，支持行选中和自定义绘制	MainDlg 中作为图书列表显示控件，在 OnInitDialog() 中设置列头（InsertColumn）和扩展样式（LVS_EX_FULLROWSELECT），通过 NM_CUSTOMDRAW 自定义行颜色
CStatic	CWnd -> CCommandTarget -> CObject	静态文本控件，用于显示标签、标题、图标等不可编辑的文本	登录页面的图标、标题、副标题、版本号；主页面的分区标题、状态栏文本、类型切换标签等，通过 OnCtlColor() 设置各自的颜色
CEdit	CWnd -> CCommandTarget -> CObject	单行或多行文本编辑框，用于接收用户输入	账号、密码、编号、书名、价格等所有输入字段；通过 ModifyStyleEx() 移除默认边框以适配深色主题
CButton	CWnd -> CCommandTarget -> CObject	标准按钮控件，支持多种样式（BS_PUSHBUTTON、	ModernButton 继承 CButton 并使用 BS_OWNERDRAW 样式实

		BS_CHECKBOX、BS_OWNERDRAW 等)	现完全自定义绘制；显示密码复选框使用 BS_CHECKBOX 样式
CComboBox	CWnd -> CCmdTarget -> CObject	下拉选择框，支持 DropList 模式（只读选择）	MainDlg 中用于选择图书类型（纸质图书/电子图书），通过 CBN_SELCHANGE 通知触发类型切换

3.3 主要类设计

本节详细分析三个最关键的 MFC 类的设计和使用方式。

CDialogEx 对话框类

CDialogEx 是 MFC 对话框的增强版本（自 VS 2010 引入），在 CDialog 基础上增加了背景色设置和图片支持。其核心机制包括：OnInitDialog() 虚函数——在对话框窗口创建完成、控件已就绪但尚未显示时调用，是初始化控件状态（如设置字体、修改样式、填充列表）的最佳时机；DoDataExchange() 虚函数——由框架自动调用，通过 DDX_Text、DDX_Control 等宏实现控件与成员变量之间的双向数据绑定；DoModal() 方法——创建模态对话框并进入内部消息循环，直到 EndDialog() 被调用才返回。本系统的 LoginDlg 和 MainDlg 均继承 CDialogEx，利用上述机制完成了界面初始化和数据交换。

CListCtrl 列表控件类

CListCtrl 封装了 Windows 的 List-View 公共控件，支持四种视图模式：Icon、Small Icon、List、Report。本系统使用 Report 视图——每行显示一条图书记录，每列显示一个字段。关键使用方式：InsertColumn() 设置列标题和宽度；InsertItem() / SetItemText() 逐行逐列填充数据；SetExtendedStyle(LVS_EX_FULLROWSELECT) 使选中高亮覆盖整行而非仅第一列；DeleteAllItems() 在刷新列表前清空全部行。此外，通过 ON_NOTIFY(NM_CUSTOMDRAW, ...) 消息映射，可以拦截列表的默认绘制过程，在每行绘制前注入自定义颜色逻辑（交替行底色、选中行高亮色），使列表控件的外观与深色主题浑然一体。

GDI 设备上下文（CDC）与自绘机制

CDC（Device Context）是 MFC 对 Windows GDI 的封装，是所有图形绘制操作的入口。本系统中涉及以下 CDC 使用场景：在 OnEraseBkgnd() 中——pDC->FillSolidRect() 逐像素绘制对话框背景的垂直渐

变（100 像素高度的头图从深色渐变到基础色）；在 `OnPaint()` 中——`dc.RoundRect()` 绘制登录页面表单卡片的圆角矩形，`dc.MoveTo()/LineTo()` 绘制账号密码输入框的下划线；在 `ModernButton::DrawItem()` 中——逐行 `FillSolidRect()` 绘制渐变填充，`RoundRect()` 绘制圆角边框，`DrawText()` 居中绘制按钮文字；在 `OnCtlColor()` 中——`pDC->SetBkMode(TRANSPARENT)` 和 `pDC->SetTextColor()` 设置 `STATIC` 控件的透明背景和文字颜色。这些 GDI 操作是实现全部 UI 自定义效果的基础。

3.4 源代码文件组织设计

MFC 项目有固定的文件组织结构，由 Visual Studio 项目模板自动生成并提供标准命名约定。本项目的 MFC 框架文件包括：

`BookManageMFC.h / BookManageMFC.cpp`：应用程序入口类 `CBookManageApp`（继承 `CWinApp`），重写 `InitInstance()` 启动登录对话框。`InitInstance()` 是 MFC 应用程序的生命周期起点——在这里创建主窗口或对话框，返回 `TRUE` 表示进入消息循环，`FALSE` 表示退出。

`BookManageMFC.rc`：资源文件，以文本形式定义所有对话框的布局模板（`DLGINIT` 段）、控件的位置和样式属性。Visual Studio 的资源编辑器提供可视化编辑，但本质上是编辑此 `.rc` 文件。包括对话框尺寸、控件坐标、标题文本、样式标志等。

`resource.h`：资源 ID 定义文件，为 `.rc` 中每个控件赋予唯一的整数 ID（如 `IDC_ACCOUNT_EDIT=1001`、`IDC_LOGIN_BUTTON=1003` 等），供 C++ 代码引用控件时使用。

`framework.h`：包含 MFC 项目的标准 Windows 头文件（`windows.h`、`afx.h`、`afxwin.h` 等），通常不需要手动修改。

`targetver.h`：定义目标 Windows 版本宏（`_WIN32_WINNT`、`NTDDI_VERSION`），控制可用的 API 集合。本系统目标为 Windows 10（`0x0A00`）。

`pch.h / pch.cpp`：预编译头文件（Precompiled Header），将不经常改动的系统头文件（如 `afxdialogex.h`、`afxwin.h`、`vector`、`map`、`string` 等）集中包含于此，利用编译器的预编译机制大幅加速增量编译（首次编译后，修改 `.cpp` 文件无需重新解析这些大型头文件）。

3.5 关键功能类及函数设计描述

以下分析 MFC 框架中几个在本系统中发挥关键作用的机制和函数。

DDX/DDV 数据交换与验证

DDX (Dialog Data Exchange) 是 MFC 中控件与成员变量之间的自动数据交换机制。在 DoDataExchange() 函数中, 使用 DDX_Text(pDX, IDC_EDIT, m_variable) 宏将编辑框控件与 CString 或数值变量绑定。框架自动在两个时机执行数据交换: OnInitDialog() 时 (变量->控件) 将初始值加载到控件显示; OnOK() 或 UpdateData(TRUE) 时 (控件->变量) 将用户输入回写到变量。DDV (Dialog Data Validation) 是对 DDX 的补充——可以在数据交换时自动验证输入范围 (如 DDV_MinMaxDouble 验证价格在 0 到某个最大值之间)。本系统在 MainDlg 中使用了 DDX 机制绑定编辑框到成员变量, 在添加和修改操作时通过 UpdateData(TRUE) 获取用户输入。

消息映射宏

MFC 的消息映射是一组宏, 将 Windows 消息 ID 映射到类的成员函数指针。核心宏包括: ON_BN_CLICKED(control_id, method)——按钮点击消息, 本系统中 8 个操作按钮和登录按钮通过此宏映射到对应的 OnBnClicked*() 处理函数; ON_NOTIFY(notify_code, control_id, method)——控件通知消息 (如 CListCtrl 的 NM_CUSTOMDRAW、CComboBox 的 CBN_SELCHANGE), 携带附加的结构体参数 (如 NMLVCUSTOMDRAW*) 提供绘图上下文信息; ON_WM_CTLCOLOR()——控件颜色消息, 在系统即将绘制子控件时调用, 允许程序返回自定义画刷 (HBRUSH) 来覆盖默认颜色; ON_WM_ERASEBKGND()——背景擦除消息, 在系统需要擦除窗口背景时调用, 程序可在此执行自定义背景绘制 (如渐变) 并返回 TRUE 告知系统无需再擦除; ON_WM_PAINT()——窗口重绘消息, 在窗口客户区需要重绘时调用, 通过 CPaintDC 在 WM_ERASEBKGND 之后执行前景绘制 (如圆角矩形、输入框下划线)。

NM_CUSTOMDRAW 自定义绘制通知

NM_CUSTOMDRAW 是 Windows 公共控件 (List-View、Tree-View、Toolbar 等) 提供的通用自定义绘制通知码。与自绘 (Owner Draw) 控件需要程序完全接管绘制不同, CUSTOMDRAW 允许程序在系统默认绘制的不同阶段插入自定义逻辑——可以仅修改颜色而不改变控件结构, 开发量远小于完全自绘。绘制分为三个阶段: CDDS_PREPAINT (绘制前, 返回 CDRF_NOTIFYITEMDRAW 请求逐项通知)、CDDS_ITEMPREPAINT (每项绘制前, 可在此修改该项的颜色)、CDDS_POSTPAINT (绘制后)。本系统在 CDDS_ITEMPREPAINT 阶段根据行号奇偶性和选中状态设置不同的背景色 (W11_ROW_ALT 用于偶数行、W11_CYAN 用于选中行), 仅改变了颜色, 行的布局和文字由系统默认绘制完成。

BS_OWNERDRAW 自绘按钮

CButton 控件的 BS_OWNERDRAW 样式指示系统: 按钮的完整绘制由程序负责, 系统仅提供绘制请求。当按钮需要重绘时, 系统向父窗口发送 WM_DRAWITEM 消息, 其中包含一个 DRAWITEMSTRUCT 结构体 (包

括设备上下文句柄 `hDC`、按钮矩形 `rcItem`、控制状态 `itemState` 等）。程序通过重写 `CButton::DrawItem()` 虚函数响应此消息，使用 `lp->hDC` 进行 GDI 绘制。本系统的 `CModernButton` 在 `DrawItem()` 中实现了五层绘制：渐变填充+圆角边框+底部阴影线+顶部光泽高光+居中文字，每种视觉状态（正常/悬停/按下）有独立的配色。这种完全自绘的方式虽然开发量较大，但实现了与深色主题完美融合的高级视觉效果。

4 新设计类功能说明

本章详细描述本项目中自行设计和实现的类——即在 MFC 框架提供的标准类之外，我们为图书管理业务专门编写的类。这些类可分为三个层次：数据层（`Book`、`PaperBook`、`EBook`）、业务逻辑层（`BookManager`）、表示层扩展（`CModernButton`、`CLoginManager`、`Colors.h`）。

4.1 总体架构设计

本系统在 MFC 对话框架构之上，采用三层架构组织自定义代码：数据层——包含 `Book` 抽象基类及其两个派生类 `PaperBook` 和 `EBook`，负责定义图书对象的数据结构和行为接口，仅依赖 C++ 标准库；业务逻辑层——以 `BookManager` 类为核心，封装 `vector<unique_ptr<Book>> + map<string,int>` 的复合数据结构，对外提供完整的 CRUD 和文件读写接口，同样不依赖 MFC；表示层扩展——在 MFC 标准控件基础上进行自定义扩展，包括 `CModernButton`（自绘按钮）、`CLoginManager`（登录验证逻辑），以及 `Colors.h`（全局颜色常量定义）。这种分层设计使得数据层和业务逻辑层可以脱离 MFC 被控制台程序复用——实际上，本项目的数据层和业务逻辑层就是从控制台版图书管理系统直接移植过来的。

4.2 类模块体系设计

以下 UML 类图展示了本系统自定义设计的核心类及其相互关系：

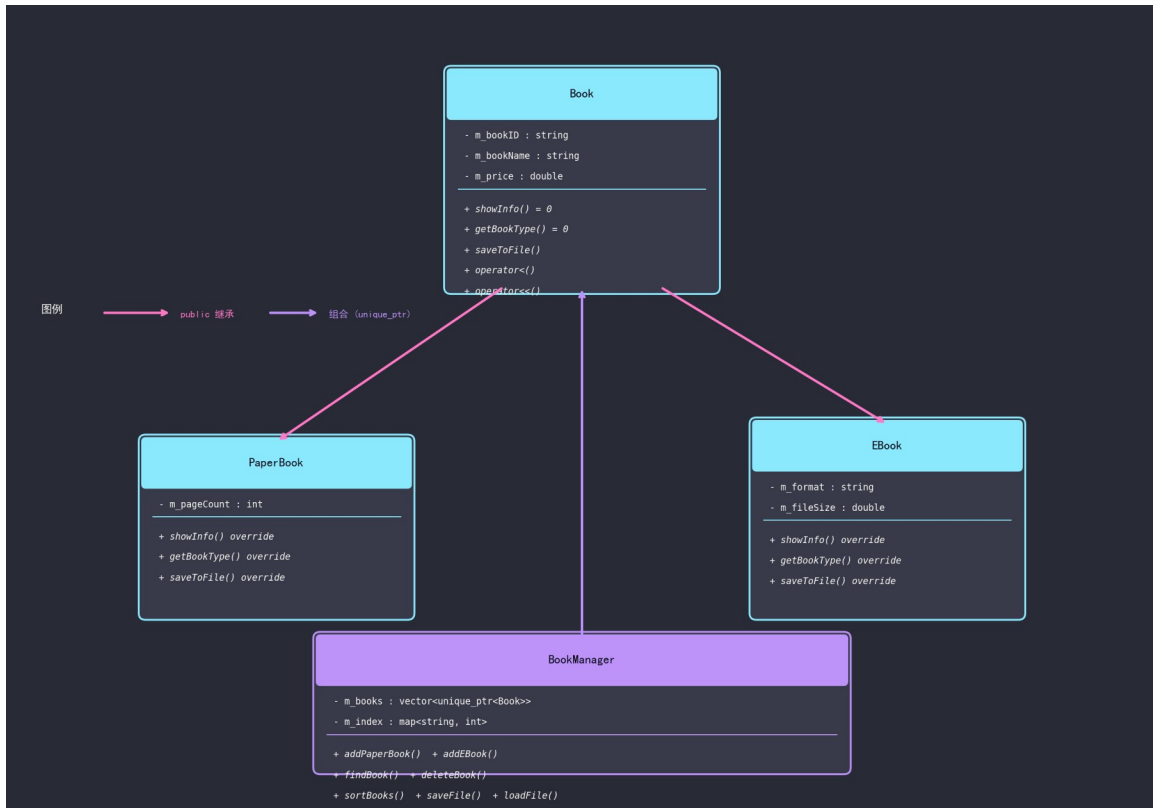


图 4-1 系统自定义核心类图

Book 是抽象基类，定义了图书对象的统一接口。三个私有成员变量 m_bookID（编号）、m_bookName（书名）、m_price（价格）被封装在类内部，外部只能通过 const 引用的 getter 和有校验的 setter 访问。两个纯虚函数 showInfo() 和 getBookType() 强制所有派生类实现自身的显示逻辑和类型标识。虚函数 saveToFile() 在基类中提供空实现，派生类按需重写——PaperBook 追加页数字段，EBook 追加格式和文件大小字段。operator< 按价格比较，用于排序；operator<< 是友元流输出运算符，格式化为可读字符串。

PaperBook 和 EBook 均以 public 方式继承 Book，各自扩展特有属性：PaperBook 增加 m_pageCount（页数，int 类型），EBook 增加 m_fileFormat（文件格式，string 类型）和 m_fileSize（文件大小，double 类型，单位 MB）。两个派生类各自重写了 showInfo()、getBookType()、saveToFile() 三个虚函数，全部使用 override 关键字标记，让编译器帮助检查签名是否与基类一致。

BookManager 作为聚合类，通过 vector<unique_ptr<Book>> 持有全部图书对象的所有权。unique_ptr 的独占所有权语义保证每个 Book 对象有且仅有一个所有者，杜绝了对象被重复释放或遗漏释放的风险。map<string,int> 维护从图书编号到 vector 下标位置的快速索引，将按编号查找的时间复杂度从 $O(n)$ 降低到 $O(\log n)$ 。rebuildIndex() 私有方法在删除或排序后重建索引。

ModernButton 继承自 CButton，使用 BS_OWNERDRAW 风格接管按钮的全部绘制。DrawItem() 中实现垂直渐变填充、圆角边框、底部阴影线和顶部光泽高光。通过 ODS_HOTLIGHT 和 ODS_SELECTED 标志区分正常、悬停和按下三种视觉状态。

4.3 数据结构类设计

Book 类主要接口

成员函数	类型	说明
Book() / Book(id, name, price)	构造函数	默认构造和带参构造，price 默认 0.0
virtual ~Book() = default	虚析构造函数	保证 unique_ptr<Book> 能正确释放派生类对象
showInfo() = 0	纯虚函数	由派生类实现各自的图书信息展示格式
getBookType() = 0	纯虚函数	返回图书类型的中文描述
saveToFile(ofstream&)	虚函数	将自身字段以竖线分隔格式写入文件流
getBookID() / getName() / getPrice()	const getter	返回 const string& 或 double 值
setBookID() / setName() / setPrice()	setter	含合法性检查：ID/书名非空检查、价格 >=0 检查
operator<	运算符重载	按 m_price 比较，供 std::sort 使用
operator<<	友元函数	流输出，格式化为 编号 书名 价格 类型

BookManager 类主要接口

成员函数	说明
addPaperBook(id, name, price, pages)	添加纸质图书：唯一性检查->make_unique<PaperBook>->push_back->更新 map
addEBook(id, name, price, fmt, fsize)	添加电子图书：同上前置检查和索引更新
findBook(id)	const 和非 const 两个重载，map O(log n) 查找，返回 Book* 或 nullptr
deleteBook(id)	按编号删除：map 定位->vector erase->rebuildIndex()
modifyBook(id, new_fields)	dynamic_cast 判类型后更新对应派生类的扩展字段
sortBooks()	std::sort + lambda 表达式按价格升序排列，排序后 rebuildIndex()
showAllBooks()	范围 for 循环调用 showInfo() 虚函数多态展示全部图书
saveFile() / loadFile()	竖线分隔文本文件读写，含头行检测和 try-catch 异常容错

isBookIDExist(id)	map 成员函数 find() 判断编号是否已存在 ($O(\log n)$)
getCount() / getBooks()	返回 m_books.size() / const vector& 引用 (仅供显示层遍历)

4.4 源代码文件组织设计

自定义类的源代码共 14 个文件，按分层组织：

数据层（6 个文件）：Book.h / Book.cpp（抽象基类定义和通用方法实现，定义纯虚函数和 setter 合法性检查），PaperBook.h / PaperBook.cpp（纸质图书派生类，扩展 m_pageCount 成员并重写所有虚函数），EBook.h / EBook.cpp（电子图书派生类，扩展 m_fileFormat 和 m_fileSize 成员并重写所有虚函数）。这组文件仅包含 C++ 标准库头文件（string、fstream、sstream、memory），不包含任何 MFC 头文件。

业务逻辑层（2 个文件）：BookManager.h / BookManager.cpp，封装 vector<unique_ptr<Book>> + map<string, int> 复合数据结构，提供全部 CRUD 操作和文件读写。同样不依赖 MFC，与控制台版图书管理系统的 BookManager 在接口上保持一致。

表示层扩展（6 个文件）：LoginDlg.h / LoginDlg.cpp（登录对话框和 CLoginManager 登录管理类，3 次尝试锁定机制），MainDlg.h / MainDlg.cpp（主管理对话框、8 个按钮的消息处理函数、SyncListFromVector 列表同步、updateExtraLabels 类型切换、OnCtlColor 控件颜色、OnEraseBkgnd 渐变背景等），ModernButton.h（BS_OWNERDRAW 自定义按钮绘制类，约 110 行），Colors.h（全局颜色常量，约 60 个宏定义，按主题色和操作按钮色分组管理）。

4.5 重点类及函数设计描述

虚析构函数与智能指针的内存安全机制

基类 Book 声明 virtual ~Book() = default。虽然析构函数体为空，但 virtual 关键字是必需的。当 vector<unique_ptr<Book>> 持有 PaperBook 或 EBook 对象时，unique_ptr 的默认删除器调用 delete ptr，此时若析构函数非虚，编译器只调用 Book::~~Book() 而跳过派生类的析构，导致派生类中的 string 成员（m_pageCount、m_fileFormat、m_fileSize 等）所占用的堆内存永远无法释放。加上

virtual 后，编译器通过 vtable 在运行时定位实际对象的析构函数，按派生类->基类的顺序逐层析构，确保所有资源正确释放。这是 C++ 多态编程中最基本也最容易出错的内存安全规则。

map 索引的维护策略 (rebuildIndex)

m_index (map<string, int>) 记录的是编号到 vector 下标的映射关系。在添加和修改操作中，已有元素位置不变，仅需在 map 中新增或更新对应条目， $O(\log n)$ 。但在删除 (erase 移除中间元素导致后续全部前移) 和排序 (元素位置完全重排) 后，map 中所有下标失效，必须调用 rebuildIndex() 清空 map 并依据 vector 当前顺序重建全部映射， $O(n)$ 。这是一种务实的权衡：以删除/排序时的 $O(n)$ 重建代价，换取查询时从 $O(n)$ 到 $O(\log n)$ 的加速。对于以查询为主的图书记录管理场景，这个选择完全合算。

竖线分隔文件的容错加载

books.txt 加载采用三层容错：第一层——首行检测 (若第一个字段以[类型]开头则为表头行，跳过；若无表头兼容旧版文件，直接从第一行解析)；第二层——字段计数 (纸质书至少 5 个字段，电子书至少 6 个，不足则跳过)；第三层——数值转换 try-catch (stod/stoi 异常时跳过当前行继续后续行)。这三层防护确保了即使文件被手动编辑损坏，程序也不会崩溃。

CModernButton 的自绘实现

继承 CButton 使用 BS_OWNERDRAW 风格，在 DrawItem() 中实现五步绘制：垂直渐变填充 (顶部叠加约 25% 亮度线性衰减至底部基准色，制造凸起感) -> RoundRect + NULL_BRUSH 圆角边框 -> 底部 1px 深色阴影线 (减 40) -> 顶部 1/3 区域半透明白色高光 (alpha 28->0) -> DrawText 居中文字。通过 ODS_HOTLIGHT/ODS_SELECTED 状态标志区分正常/悬停/按下三套配色。

NM_CUSTOMDRAW 列表自定义行色

在 OnNMCustomdrawList() 中处理 CDDS_ITEMPREPAINT 阶段：对偶数行设置 W11_ROW_ALT 交替底色，选中行覆盖 W11_CYAN 高亮+BTN_TEXT 深色文字。仅修改颜色，行的布局和文字仍由系统默认绘制完成，是定制程度与开发成本之间很好的平衡点。

CLoginManager 的独立封装

登录逻辑独立于 UI 封装在 CLoginManager 类中：validate(account, password) 验证凭证，fail() 递增失败计数，locked() 判断 3 次上限，left() 返回剩余机会。LoginDlg 只需根据三个返回值 (成

功/失败/锁定) 决定 UI 行为, 验证逻辑可被独立测试和修改。这种分离比教师参考示例中直接在按钮回调里硬编码验证的做法有明显改进。

5 部署运行和使用说明

5.1 编译安装

本系统提供两种使用方式: 直接运行预编译的 exe 文件, 或在 Visual Studio 中编译源码。

方式一: 直接运行 (推荐)

将 BookManageMFC.exe_ 重命名为 BookManageMFC.exe (PTA 平台不允许上传 .exe 文件, 故提交时后缀改为 .exe_), 双击即可运行。无需安装任何额外运行时库——程序已使用 MFC 静态链接 (/MT), 运行时库嵌入在 exe 中。

方式二: 从源码编译

使用 Visual Studio 2022 或 2026 打开 BookManageMFC.sln。如果编译报错 MSB8020 (找不到 v145 工具集), 右键项目->属性->配置属性->常规->平台工具集, 改为 v143 (VS 2022 自带 v143) 或保持 v145 (VS 2026 自带 v145)。确保在 VS Installer 中已勾选[C++ MFC for v14xx build tools]。配置选择 Release x64, 按 Ctrl+Shift+B 生成解决方案。编译完成后在 x64/Release 目录下找到 BookManageMFC.exe。

5.2 运行测试

为验证系统各项功能是否正常工作, 建议按以下顺序进行运行测试:

(1) 登录测试: 输入错误密码, 验证提示信息 and 剩余次数; 输入正确账号 admin 密码 book2026, 验证能否进入主界面。连续 3 次错误, 验证程序是否锁定退出。

(2) 添加测试: 分别添加纸质图书 (如编号 B001、书名《C++程序设计》、价格 59.00、页数 350) 和电子图书 (如编号 E001、书名《算法导论》、价格 89.00、格式 PDF、文件大小 12.5), 确认列表中正确显示。尝试重复添加同一编号, 验证是否提示编号已存在。

(3) 查询测试：在编号框中输入已存在的编号（如 B001），点击查询，验证列表是否高亮选中对应行，右侧编辑区是否自动填充完整信息。输入不存在的编号，验证是否提示未找到。

(4) 修改测试：查询到某图书记录后，将价格改为另一数值，点击修改，验证列表和内存数据是否同步更新。切换图书类型，验证附加字段标签是否正确切换。

(5) 删除测试：在列表选中某行，点击删除，在确认弹窗中选择[是]，验证该记录是否从列表中消失。再次查询该编号，验证确实无法找到。

(6) 排序测试：点击排序按钮，验证列表中记录是否按价格升序重新排列。排序完成后执行一次查询，验证 map 索引是否正确重建。

(7) 保存与加载测试：添加几条记录后点击保存，关闭程序重新启动，验证之前添加的数据是否自动恢复显示。

(8) 异常输入测试：尝试将价格填为负数、编号留空、页数填 0，验证系统是否正确弹窗提示。

5.3 使用操作

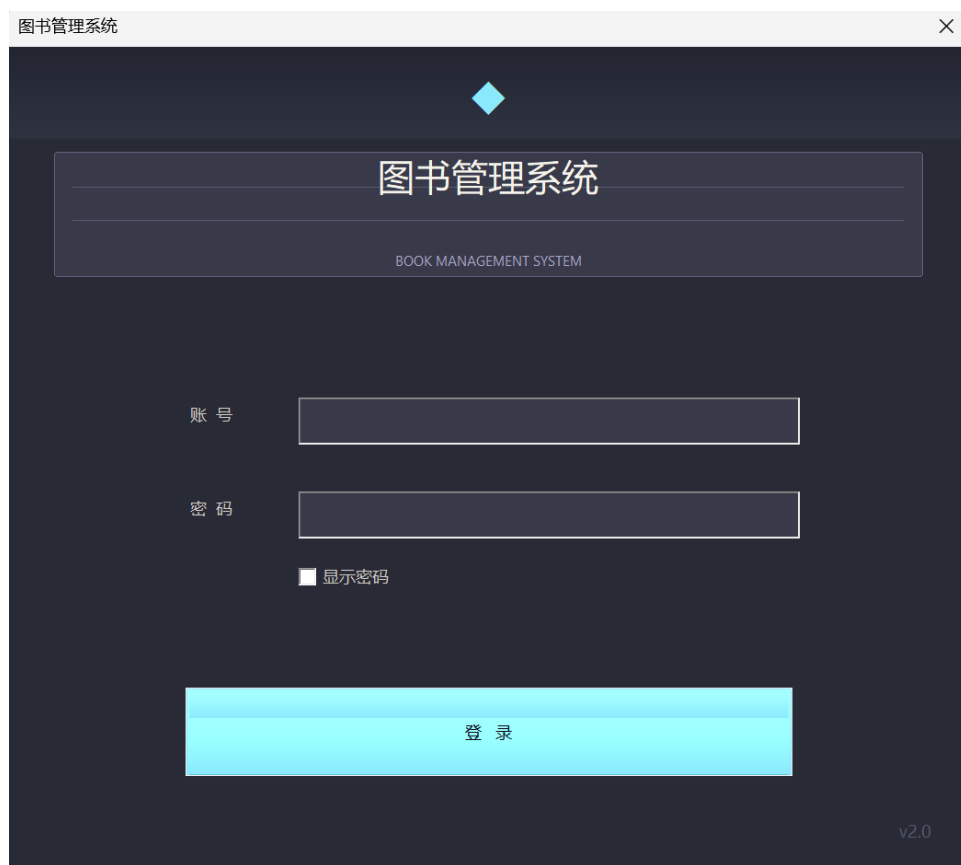


图 5-1 登录界面

图 5-1 展示了系统的登录界面。顶部是菱形图标和系统标题[图书管理系统]，中间是带圆角的深色表单卡片，卡片内包含账号输入框、密码输入框和青色登录按钮，下方有显示密码复选框和版本号。整个界面采用深色渐变背景——顶部约 100 像素由深变浅的渐变头图，与主界面风格保持一致。所有文本控件（标题、副标题、版本号）使用不同的颜色等级来建立视觉层次。

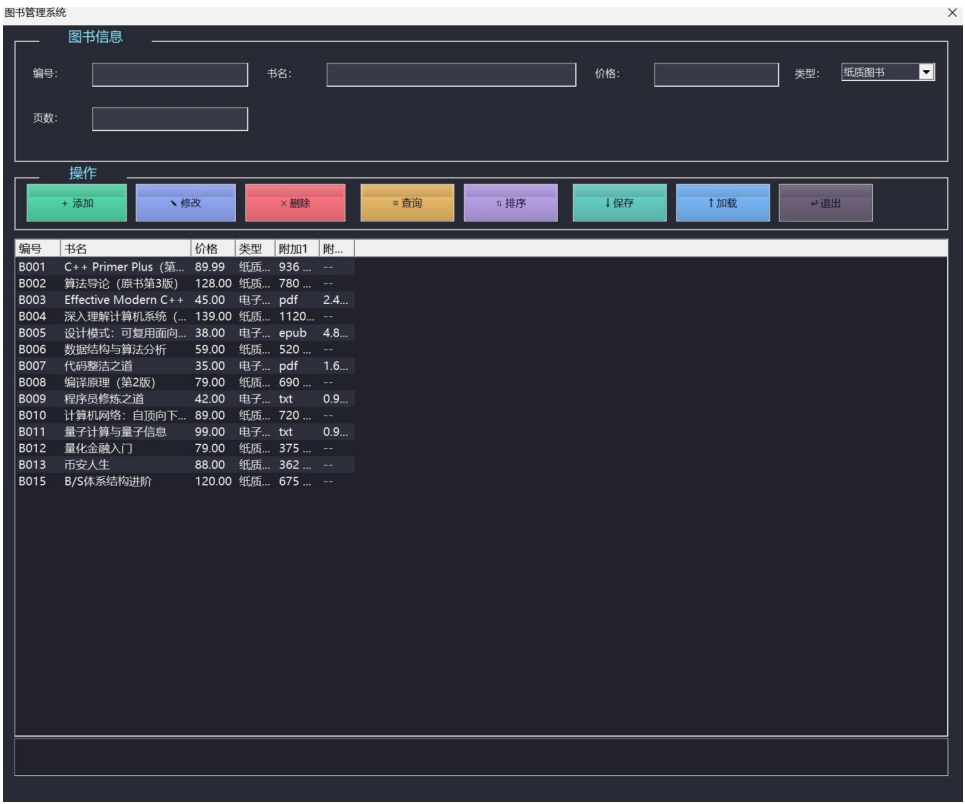


图 5-2 主界面（含示例数据）

图 5-2 展示了系统的主管理界面。左侧列表控件以 Report 视图展示全部图书记录（编号、书名、价格、类型、页数/格式、文件大小），偶数行和奇数行有不同的背景色深度，选中行以青色高亮。右侧上方为[图书信息]编辑区，下方为 8 个彩色操作按钮。状态栏显示当前图书记录数。

图书管理系统

×

图书信息

编号:

书名:

价格:

类型: 纸质图书

页数:

操作

+ 添加

修改

× 删除

= 查询

排序

↓ 保存

↑ 加载

退出

编号	书名	价格	类型	附加1	附...
B001	C++ Primer Plus (第...	89.99	纸质...	936 ...	--
B002	算法导论 (原书第3版)	128.00	纸质...	780 ...	--
B003	Effective Modern C++	45.00	电子...	pdf	2.4...
B004	深入理解计算机系统 (...	139.00	纸质...	1120...	--
B005	设计模式: 可复用面向...	38.00	电子...	epub	4.8...
B006	数据结构与算法分析	59.00	纸质...	520 ...	--
B007	代码整洁之道	35.00	电子...	pdf	1.6...
B008	编译原理 (第2版)	79.00	纸质...	690 ...	--
B009	程序员修炼之道	42.00	电子...	txt	0.9...
B010	计算机网络: 自顶向下...	89.00	纸质...	720 ...	--
B011	量子计算与量子信息	99.00	电子...	txt	0.9...
B012	量化金融入门	79.00	纸质...	375 ...	--
B013	市安人生	88.00	纸质...	362 ...	--
B015	B/S体系结构进阶	120.00	纸质...	675 ...	--
B020	离散数学及其应用	58.00	纸质...	700 ...	--

已添加

图 5-3 添加图书操作

图 5-3 展示了添加图书的过程。类型下拉框选择[纸质图书]后，附加字段自动切换为[页数]标签和输入框；选择[电子图书]则切换为[格式]和[文件大小]。填写编号、书名、价格、页数后点击绿色[添加]按钮，新记录出现在列表末尾，状态栏提示添加成功。

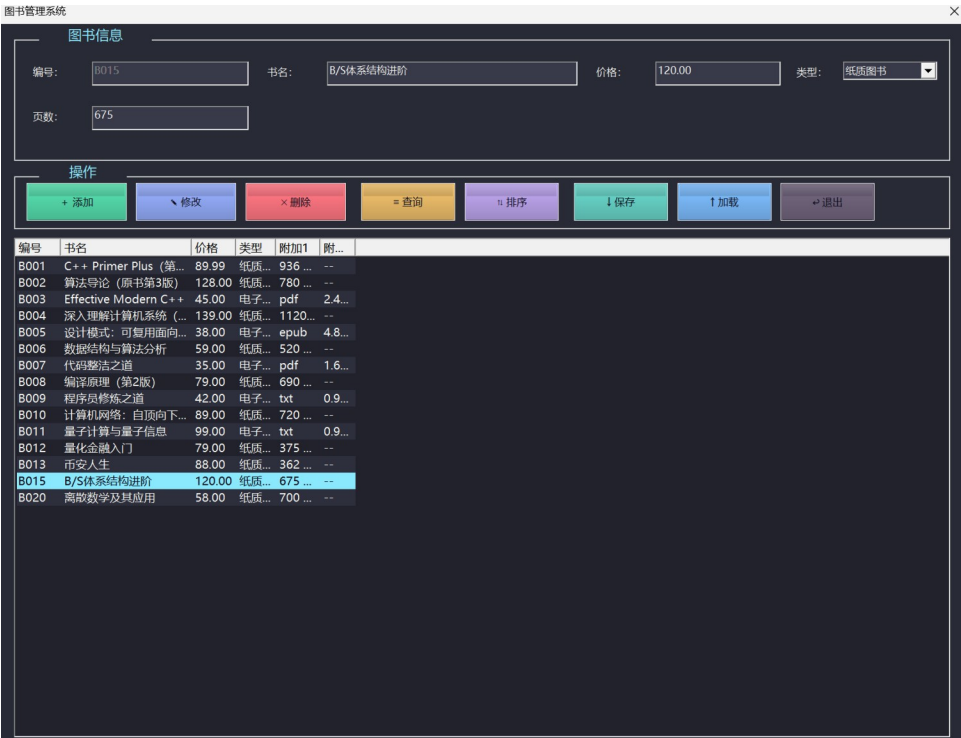


图 5-4 查询与修改操作

图 5-4 展示了查询和修改功能。在编号输入框中输入要查找的图书编号，点击金色[查询]按钮后，列表中对应行自动高亮选中，右侧编辑区自动填充该图书记录的全部信息。用户可直接在编辑框中修改字段值，修改完成后点击蓝紫色[修改]按钮提交更新，列表和状态栏同步刷新。

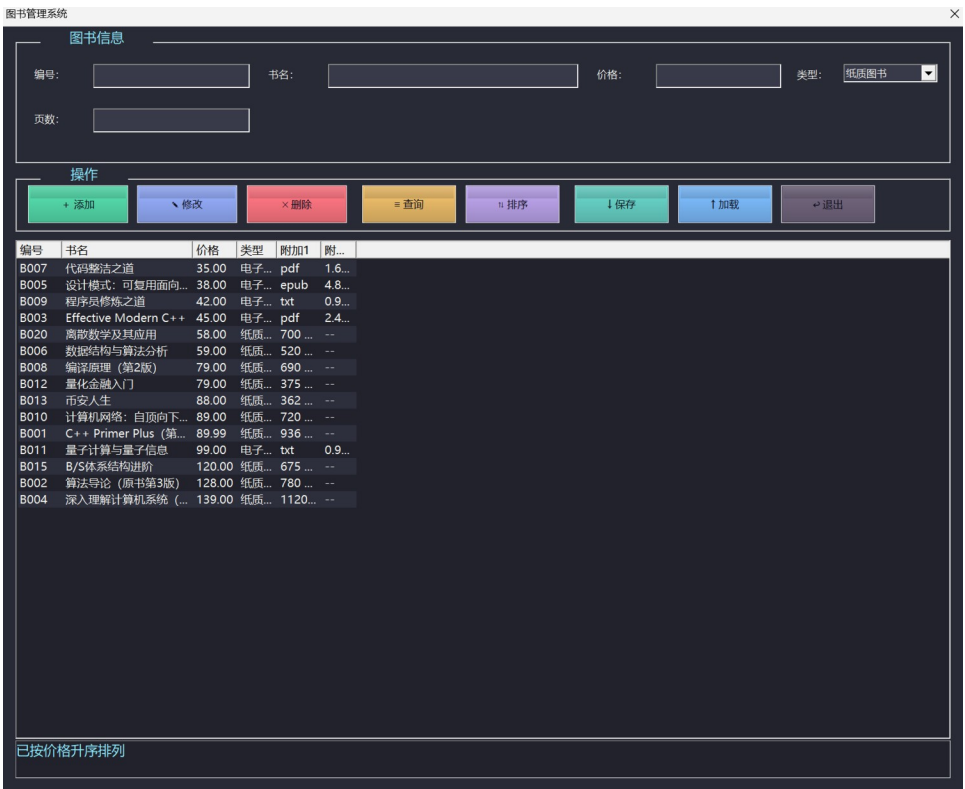


图 5-5 排序后的列表状态

图 5-5 展示了按价格排序后的效果。点击淡紫色[排序]按钮后，CListCtrl 中的全部记录按价格升序重新排列（从最低价到最高价），状态栏显示[排序完成]提示。排序后 map 索引已自动通过 rebuildIndex() 重建，后续的查询操作不受影响。

5.4 收获感言

通过本项目的完整开发——从需求分析、类设计、编码实现、调试优化到文档撰写——我对面向对象程序设计的核心概念有了从课本上看到到代码里用起来的实质性理解。

在封装方面，Book 类的三个私有成员变量让我体会到信息隐藏的好处：外部代码只能通过 getter/setter 访问数据，setter 中的合法性检查成为数据天然的守门员。任何想写入非法值的地方都会在入口处被拦截，而不是事后补救。

在继承和多态方面，Book 抽象基类定义纯虚函数 showInfo() = 0，PaperBook 和 EBook 各自重写，而 BookManager 只管用基类指针调用 bp->showInfo()，完全不用关心具体是纸质的还是电子的。这种设计在添加新图书类型时特别有用——只需新增一个派生类并实现那几个虚函数，BookManager 的所有遍历代码一行不用改。modifyBook() 中 dynamic_cast 判断类型的做法让我第一次在实践中用到了 RTTI。

在 MFC 开发方面，通过自定义 BS_OWNERDRAW 按钮和 NM_CUSTOMDRAW 列表行渲染，我理解了 GDI 图形绘制的基本流程和 Windows 消息驱动模型。OnEraseBkgnd 与 OnPaint 的调用顺序、STATIC 控件的 TRANSPARENT 背景机制导致的文本重叠问题、GROUPBOX 与其他控件的 Z-order 视觉层级——这些都是在书本上学不到但实际开发中非常具体且有挑战性的知识点。

在 STL 使用方面，`vector<unique_ptr<Book>> + map<string,int>` 的组合设计解决了既要保持元素顺序又要快速按 ID 查找的矛盾。`rebuildIndex()` 的引入让我认识到实际工程中的数据结构设计往往是一种权衡——用删除/排序后多花 $O(n)$ 重建索引换取查询从 $O(n)$ 降到 $O(\log n)$ 。

总的来说，这个项目让我完整经历了一个 C++ 桌面应用从零到交付的全过程。虽然踩了不少坑——STATIC 控件文本重叠调试了整整一个下午，GROUPBOX 与按钮的位置对齐反复调整了五六次才满意——但每个坑踩过之后对 Windows 消息机制和 GDI 渲染模型的理解都实实在在地进了一步。这种从实践中学习的收获，是仅仅阅读教材或听课无法获得的。

6 参考文献资料

以下为本项目开发过程中参考的主要资料，均为可公开访问或正式出版的资源：

- [1] Microsoft Learn. MFC Desktop Applications [EB/OL].
<https://learn.microsoft.com/en-us/cpp/mfc/mfc-desktop-applications>, 2024.
- [2] Microsoft Learn. CListCtrl Class [EB/OL].
<https://learn.microsoft.com/en-us/cpp/mfc/reference/clistctrl-class>, 2024.
- [3] Microsoft Learn. NM_CUSTOMDRAW (ListView) Notification Code [EB/OL].
<https://learn.microsoft.com/en-us/windows/win32/controls/nm-customdraw-list-view>, 2024.
- [4] Jeff Prosise. Programming Windows with MFC, 2nd Edition [M]. Microsoft Press, 1999. ISBN 1-57231-695-0.
- [5] Stanley B. Lippman, Josee Lajoie, Barbara E. Moo. C++ Primer, 5th Edition [M]. Addison-Wesley, 2012. ISBN 978-0-321-71411-4.
- [6] Bjarne Stroustrup. The C++ Programming Language, 4th Edition [M]. Addison-Wesley, 2013. ISBN 978-0-321-56384-2.

- [7] Scott Meyers. Effective Modern C++: 42 Specific Ways to Improve Your Use of C++11 and C++14 [M]. O'Reilly Media, 2014. ISBN 978-1-491-90399-5.
- [8] cppreference.com. C++ Standard Library Reference [EB/OL]. <https://en.cppreference.com/w/>, 2024.
- [9] Norman Yee (Software Kinetics Ltd). Extended Use of CStatic Class -- CLabel 1.6 [EB/OL]. CodeProject. <https://www.codeproject.com/Articles/215/Extended-Use-of-CStatic-Class-CLabel-1-6>, 2001.
- [10] Dracula Theme. Official Color Palette [EB/OL]. <https://draculatheme.com>, 2024.